

从选手的角度来看 LMCC 第二轮试题

常见知识点及简明做题方法

Peiqi Gao

浙江海亮高级中学

2026 年 8 月 12 日

Outline

观前提示

何为选手视角

LMCC 第二轮常见知识点梳理

Prompt 编写方法

生成参数配置

Python 代码实现

尾声



观前提示

本 PPT 的目标，是低门槛地，让大家在目前的 LMCC 第二轮的题目架构内拿到高分。
所以几乎零基础、只知道一点点基本概念的同学亦可食用本课件！



观前提示

本 PPT 的目标，是**低门槛地**，让大家在目前的 **LMCC 第二轮的题目架构内拿到高分**。
所以几乎零基础、只知道一点点基本概念的同学亦可食用本课件！

其中，Prompt 和生成参数部分是**几乎零基础**的（但需要先了解 Prompt 是什么、思考模式是什么……这些第一轮串讲应该提过，不清楚的话也可以现场查一下）；
Python 部分可参考其他老师的讲解，我们**也会给出额外的关于学习方法和路线的建议**。

这个 PPT 的很多内容都是自洽的，所以大家完全可以主要参考该 PPT 来准备 LMCC。
(特别地，Prompt 部分)

Outline

观前提示

何为选手视角

LMCC 第二轮常见知识点梳理

Prompt 编写方法

生成参数配置

Python 代码实现

尾声



何为选手视角

本 PPT 中的**选手视角**，是以**实际的竞赛（例如 LMCC 第二轮）**为目标的。即**提炼**训练题带来的**经验**，**总结**出一些在竞赛中简单实用的**技巧**，并**更快更轻松地在考场上切题**。即，“以真题为镜，以复用为本，以效率为终。”



何为选手视角

本 PPT 中的**选手视角**，是以**实际的竞赛（例如 LMCC 第二轮）**为目标的。即**提炼**训练题带来的**经验**，**总结**出一些在竞赛中简单实用的**技巧**，并**更快更轻松地在考场上切题**。即，“以真题为镜，以复用为本，以效率为终。”

接下来，我们会将**相对抽象且看似“小作文”型的 Prompt 编写任务**转为**利用六字口诀，对文本的摘抄与微调任务**，降低大家学习的门槛；分析那些**生成参数**中常用的**参数**，并提供**简明、一页纸的设定参数妙招**；最后，我们还会**以真实题目为例**，来提供**torch 系列函数的助记技巧**。

PS：为求完整，本 PPT 的文本量可能会有点大。
如果大家有“晕字”的情况，可以优先看**加粗**的内容。

Outline

观前提示

何为选手视角

LMCC 第二轮常见知识点梳理

Prompt 编写方法

生成参数配置

Python 代码实现

尾声



我没有任何基础！LMCC 第二轮的每道题，本质上在要求我干什么？

本质上，它要求你将一个名为 `submission.py` 的程序补充完整（例如，补充完整一个数学计算助手的程序，就需要编写计算接口和提示词两部分）。随后，评测系统会在 `evaluate.py` 中将一些输入喂给 `submission.py`，而你需在此过程中争取尽可能高的分数。

从选手角度，梳理 LMCC 第二轮常见知识点

本 PPT 将选用 **六道题（A1、A2、B1、B2、C1、C2）** 作为例题。
它们来自三个训练题集：**A**（260124）、**B**（260125）、**C**（LMCC-2025 第二轮真题）。
课后我们会提供相关文件，内含这些题目的详细资料，供大家自行查阅。



从选手角度，梳理 LMCC 第二轮常见知识点

本 PPT 将选用 **六道题（A1、A2、B1、B2、C1、C2）** 作为例题。

它们来自三个训练题集：**A**（260124）、**B**（260125）、**C**（LMCC-2025 第二轮真题）。

课后我们会提供相关文件，内含这些题目的详细资料，供大家自行查阅。

六道例题所需的知识点分布是如下表所示的。因此，我们的讲解将围绕这三个模块展开：**Prompt 编写方法（重点）、生成参数配置、代码实现**（以 A2 的 torch 为例）。

题目	Prompt 编写方法	生成参数配置	Python 代码实现
A1 作业批改	● ● ●	● ●	
A2 硬件 RAG	● ●	●	● ● ●
B1 三国知识提取	● ● ●	● ●	
B2 计算器工具调用	● ● ●	●	● ● ●
C1 古诗标准化	● ● ●	● ●	
C2 古诗 RAG	● ●	●	● ● ●

Outline

观前提示

何为选手视角

LMCC 第二轮常见知识点梳理

Prompt 编写方法

生成参数配置

Python 代码实现

尾声



提示词设计之“卫星隔离不明”，快速写出高分 Prompt

下面为大家介绍一个笔者总结的 Prompt 编写方法。经典、简单、好用，且专门针对 LMCC 竞赛设计。不需要高水平，**只需具备分类信息的能力和一颗勇于调试的心。**透过此法，你不仅能**几乎零基础地快速上手赛时的 Prompt 编写（赛时我们只需编写约 30% 的提示词内容，剩下照抄题面即可）**，还能顺带梳理其中包含的关键元素。



提示词设计之“卫星隔离不明”，快速写出高分 Prompt

下面为大家介绍一个笔者总结的 Prompt 编写方法。经典、简单、好用，且专门针对 LMCC 竞赛设计。不需要高水平，**只需具备分类信息的能力和一颗勇于调试的心。**

透过此法，你不仅能**几乎零基础地快速上手赛时的 Prompt 编写（赛时我们只需编写约 30% 的提示词内容，剩下照抄题面即可）**，还能顺带梳理其中包含的关键元素。

对于这个方法，我们称之为“**卫星隔离不明**”。

（**骨架**：谐音 **位行格例**，即身份定**位**、**行**动流程、输出**格**式和标准示**例**）

（为便于指令遵循，笔者一般在 Prompt 中把“**行动流程**”写作“**任务要求**”）

（其中的 **不明** 则是“**命令**”（**明**）与“**补充**”（**不**）的意思，是 **修饰**）

提示词设计之“卫星隔离不明”，快速写出高分 Prompt

下面为大家介绍一个笔者总结的 Prompt 编写方法。经典、简单、好用，且专门针对 LMCC 竞赛设计。不需要高水平，**只需具备分类信息的能力和一颗勇于调试的心。**

透过此法，你不仅能**几乎零基础地快速上手赛时的 Prompt 编写（赛时我们只需编写约 30% 的提示词内容，剩下照抄题面即可）**，还能顺带梳理其中包含的关键元素。

对于这个方法，我们称之为“**卫星隔离不明**”。

（**骨架**：谐音 **位行格例**，即身份**定位**、**行动**流程、输出**格式**和标准**示例**）

（为便于指令遵循，笔者一般在 Prompt 中把“**行动流程**”写作“**任务要求**”）

（其中的 **不明** 则是“**命令**”（**明**）与“**补充**”（**不**）的意思，是 **修饰**）

这个想法其实很自然，因为在基本的 Prompt 编写流程中，我们主要做的也就是写下一句句形如“你是一个……助手”（**身份定位**）、“从输入的文本里提取……”（**行动流程**）、“输出纯文本……”（**输出格式**）、“不能漏掉任何一个！”（**命令**）、“不论故事中主要或非主要……”（**补充**）的文本。这些文本均可归入这六大类别之一。

“卫星隔离不明”下，赛时 Prompt 的大致框架

大致框架如下。具体而言，开头的**身份定位**部分，你需要为 AI **赋予身份**（如“严格的作业批改助手”），并**一句话总述其任务要求**（如“你的任务是读取学生的作业文本，判断每道题答案的正确性，并输出标准 JSON”）；在**【任务要求】**部分，你需要**按顺序列出任务流程**；在**【输出格式】**部分，你需要明确输出格式；在**【标准示例】**部分，**一般我们只列出一个示例（Few-Shot）供 AI 参考**。示例数量可酌情增加。

你是一个xxx助手。xxx

【任务要求】

1. xxx
2. xxx
3. xxx

【输出格式】

xxx

【标准示例】

xxx（在上述的“xxx”的地方写“命令”与“补充”）

站在选手的角度，如何在赛时快速写出优秀的 Prompt?

我们先一句话概括核心任务：**分类、整理提示中的信息，让 Prompt 变得简洁完整，并在编写和调试过程中利用命令与补充来完善 Prompt。**



站在选手的角度，如何在赛时快速写出优秀的 Prompt?

我们先一句话概括核心任务：**分类、整理提示中的信息，让 Prompt 变得简洁完整，并在编写和调试过程中利用命令与补充来完善 Prompt。**

接下来，我们详细说说此法的完整流程：

1. **整理归类** 题面/注释中的提示文本所包含的 **有效信息**。
2. 将 **同类的信息放在一起**，并按照 Prompt 的大致框架（例如我们将要介绍的）组合这些话。调整语言，删去意义重复的话，**让 Prompt 在完整的前提下保持简洁**。
3. 思考 Prompt 可能的漏洞。适当增加一些 **命令/补充**，**让 Prompt 变得完整**。
4. 此时 Prompt 可能还比较平淡。试着增添少许 **命令**，**严肃地强调若干关键点**。
5. 测试你的程序，并根据程序在题目上输出的错误结果进行调整，即 **debug**。

直接运用这个流程，大家**就可以较为顺利地编写出优秀的 Prompt 了**。初看此流程占据了大半页 PPT，但实则正如“解”字的流畅写法、信息学竞赛中的“A+B Problem”一般，稍加练习即可信手拈来。故**建议大家在课后实操至少两个 Prompt**。

“命令”和“补充”分为哪几大类？如何写好它们？

1. “命令”（建立权威）

大模型需要清晰的边界感和语气上的强制力：

- **定身份**：在 **身份定位** 中**开门见山点明角色**。
- 范例：你是一个严格的作业自动批改助手。
- **禁止句 / 语气严肃句**：使用**否定词和严肃的限制词**，防止模型输出多余内容。
- 关键词：绝对不要……!；仅……；严禁……
- 范例：绝对不要包含 ``json 等 Markdown 格式，严禁输出任何解释或多余字符！

2. “补充”（堵住漏洞）

- **补充细节 / 边界情况（Corner Case）**：针对可能干扰模型的文本**添加防御**。
- 范例：忽略作业中的解题过程或无关文字说明（防止模型被学生的过程带偏）。
- **被轻描淡写的关键内容**：**字面含义**，会在例题中遇到。

例题

接下来，我们以 **A1**、**A2**、**B1**、**B2** 为例，实际演示 Prompt 的编写过程。

课后，大家可以继续用此方法尝试编写 **C1**、**C2** 的 Prompt，我们同样附上了参考答案，方便大家比对。

例题：A1-数学作业批改助手

原题题面

考生实现：定义 `system prompt`

要求：

1. 引导模型理解作业批改任务，读取学生作业内容，判断答案是否正确
2. 说明输入是学生作业文件内容，输出是标准 JSON
3. 引导模型从作业文件中正确识别出：
 - 每道题的学生答案
 - 每道题的正确性判断 (`true/false`)
4. 要求模型严格按照标准 JSON 格式输出（包含 `student_id`、`judgements` 两个字段）
5. 强调要识别答案的核心内容，忽略无关的文字说明
6. 提供多个示例帮助模型理解任务

标准 JSON 格式：

```
{  
  "student_id": "学生 ID",  
  "judgements": [true, false, true, true, false]  
}
```

- `'judgements'` 字段：
 - 必须是布尔值列表
 - 长度必须为 5（对应 5 道题）
 - 每个元素必须是 `'true'` 或 `'false'`
- 必须是有效的 JSON 格式（可被 `'json.loads()'` 解析）
- **不要用 markdown 代码块包裹**（不要用 `\\`\\`json` 或 `\\`\\``）**
- 直接输出纯 JSON 字符串，不要添加任何说明文字

个人实现

例题：A1-数学作业批改助手

原题题面

考生实现：定义 system prompt

要求：

1. 引导模型理解作业批改任务，读取学生作业内容，判断答案是否正确
2. 说明输入是学生作业文件内容，输出是标准 JSON
3. 引导模型从作业文件中正确识别出：
 - 每道题的学生答案
 - 每道题的正确性判断 (true/false)
4. 要求模型严格按照标准 JSON 格式输出 (包含 student_id、judgements 两个字段)
5. 强调要识别答案的核心内容，忽略无关的文字说明
6. 提供多个示例帮助模型理解任务

标准 JSON 格式：

```
{
  "student_id": "学生 ID",
  "judgements": [true, false, true, true, false]
}
```

- 'judgements' 字段：
 - 必须是布尔值列表
 - 长度必须为 5 (对应 5 道题)
 - 每个元素必须是 'true' 或 'false'
 - 必须是有效的 JSON 格式 (可被 'json.loads()' 解析)
 - **不要用 markdown 代码块包裹** (不要用 '\\`json 或 '\\`')
 - 直接输出纯 JSON 字符串，不要添加任何说明文字

个人实现

你是一个严格的作业自动批改助手。你的任务是读取学生的作业文本，判断每道题答案的正确性，并输出标准 JSON。

【任务要求】

1. 从输入的作业文本中提取 **student_id** 和每道题的判断结果。
2. 忽略 **作业中的解题过程** 或无关文字说明，**仅核对核心答案**。
3. 判断结果为布尔值：正确为 **true**，错误为 **false**。

【输出格式】

严格输出纯 JSON 字符串，包含 **student_id** 和 **judgements** 两个字段。
绝对不要包含 ``json 等 Markdown 格式，不要输出**任何解释或多余字符**！

【标准 JSON 示例】

输入：学生 ID: 202401，第一题答案 5，第二题答案 x=3
输出：{"student_id": "202401", "judgements": [true, false]}

本地测试分数：50/50

例题：A1-数学作业批改助手

原题题面

考生实现：定义 system prompt

要求：

1. 引导模型理解作业批改任务，读取学生作业内容，判断答案是否正确
2. 说明输入是学生作业文件内容，输出是标准 JSON
3. 引导模型从作业文件中正确识别出：
 - 每道题的学生答案
 - 每道题的正确性判断 (true/false)
4. 要求模型严格按照标准 JSON 格式输出 (包含 student_id、judgements 两个字段)
5. 强调要识别答案的核心内容，忽略无关的文字说明
6. 提供多个示例帮助模型理解任务

标准 JSON 格式：

```
{
  "student_id": "学生 ID",
  "judgements": [true, false, true, false]
}
```

- `judgements` 字段：
 - 必须是布尔值列表
 - 长度必须为 5 (对应 5 道题)
 - 每个元素必须是 `true` 或 `false`
 - 必须是有效的 JSON 格式 (可被 `json.loads()` 解析)
 - ****不要用 markdown 代码块包裹**** (不要用 `\\`json 或 `\\`)
 - 直接输出纯 JSON 字符串，不要添加任何说明文字

个人实现

你是一个严格的作业自动批改助手。你的任务是读取学生的作业文本，判断每道题答案的正确性，并输出标准 JSON。

【任务要求】

1. 从输入的作业文本中提取 **student_id** 和每道题的判断结果。
2. 忽略 **作业中的解题过程** 或无关文字说明，**仅核对核心答案**。
3. 判断结果为布尔值：正确为 **true**，错误为 **false**。

【输出格式】

严格输出纯 JSON 字符串，包含 **student_id** 和 **judgements** 两个字段。
绝对不要包含 ``json 等 Markdown 格式，不要输出**任何解释或多余字符**！

【标准 JSON 示例】

输入：学生 ID: 202401，第一题答案 5，第二题答案 $x=3$
输出：{"student_id": "202401", "judgements": [true, false]}

本地测试分数：50/50

“命令”：

1. “你是一个……助手”：**定身份**
2. “绝对不要……！”，“仅”：**禁止/语气严肃句**

“补充” (部分内容可选，后同)：

1. “student_id”：**被轻描淡写的关键内容**
2. “作业中的解题过程”，“任何解释或多余字符”：**加细节**

例题：A2-硬件知识问答助手

原题题面

考生实现：组装用户消息（包含检索内容和题目）

参数：

`problem`: 当前要解答的题目

`retrieved_problems`: 检索到的相似条目列表，每个元素包含 `id`, `knowledge` 字段

返回：

完整的用户消息字符串

考生实现：定义 `system prompt`（RAG 版本）

要求：

1. 引导模型理解硬件选择题的任务
2. 说明会提供检索到的相似条目作为参考
3. 要求模型以 JSON 格式输出，包含 `reasoning` 和 `answer` 两个字段
4. `answer` 字段只包含选项字母（A/B/C/D）

模型必须输出标准的 JSON 格式：

```
```json
{
 "reasoning": "回答选择题的依据",
 "answer": "A"
}
```
```

- `reasoning` 字段：说明选择理由
- `answer` 字段：只包含选项字母（A/B/C/D）
- 必须是有效的 JSON 格式（可被 `json.loads()` 解析）
- **不要用 markdown 代码块包裹**（不要用 ````json` 或 `````）
- 直接输出纯 JSON 字符串，不要添加任何说明文字

个人实现

例题：A2-硬件知识问答助手

原题题面

考生实现：组装用户消息（包含检索内容和题目）

参数：

`problem`: 当前要解答的题目

`retrieved_problems`: 检索到的相似条目列表，每个元素包含 `id`, `knowledge` 字段

返回：

完整的用户消息字符串

考生实现：定义 `system prompt`（RAG 版本）

要求：

1. 引导模型理解硬件选择题的任务
2. 说明会提供检索到的相似条目作为参考
3. 要求模型以 JSON 格式输出，包含 `reasoning` 和 `answer` 两个字段
4. `answer` 字段只包含选项字母（A/B/C/D）

模型必须输出标准的 JSON 格式：

```
```json
{
 "reasoning": "回答选择题的依据",
 "answer": "A"
}
```
```

- `reasoning` 字段：说明选择理由
- `answer` 字段：只包含选项字母（A/B/C/D）
- 必须是有效的 JSON 格式（可被 `json.loads()` 解析）
- ****不要用 markdown 代码块包裹****（不要用 ````json` 或 `````）
- 直接输出纯 JSON 字符串，不要添加任何说明文字

个人实现

`{retrieved_knowledge}`

请基于以上检索到的硬件知识，回答以下选择题：

题目：{`problem`}

请严格遵循以下要求：

1. **仔细阅读题目和检索到的知识**
2. 基于检索到的知识进行**推理**
3. 输出格式**必须是**标准的 JSON，包含 `reasoning` 和 `answer` 两个字段
4. `answer` 字段只能包含选项字母（A/B/C/D），**不能有其他字符**

你是一个严格的计算硬件知识问答助手。你的任务是根据检索到的相似硬件知识条目，解答**计算**硬件相关的单项选择题，并输出标准 JSON。

【任务要求】

1. 参考提供的相似硬件知识条目解释，**分析**并回答当前硬件选择题。
2. `reasoning` 字段：说明选择理由和判断依据。
3. `answer` 字段：仅包含**正确**的选项字母（A/B/C/D），不要包含任何其他字符。

【输出格式】

严格输出纯 JSON 字符串，必须包含 `reasoning` 和 `answer` 两个字段。
绝对不要包含 ````json` 或 ````` 等 Markdown 代码块，不要输出任何解释或多余字符！ 【标准 JSON 示例】

```
{
  "reasoning": "GPU 擅长进行大规模并行计算，常用于图形渲染和并行任务，因此选择 A。",
  "answer": "A"
}
```

本地测试分数：50/50

例题：A2-硬件知识问答助手

原题题面

考生实现：组装用户消息（包含检索内容和题目）

参数：

`problem`: 当前要解答的题目

`retrieved_problems`: 检索到的相似条目列表，每个元素包含 `id`, `knowledge` 字段

返回：

完整的用户消息字符串

考生实现：定义 `system prompt`（RAG 版本）

要求：

1. 引导模型理解硬件选择题的任务
2. 说明会提供检索到的相似条目作为参考
3. 要求模型以 `JSON` 格式输出，包含 `reasoning` 和 `answer` 两个字段
4. `answer` 字段只包含选项字母（A/B/C/D）

模型必须输出标准的 `JSON` 格式：

```
```json
{
 "reasoning": "回答选择题的依据",
 "answer": "A"
}
```
```

- `reasoning` 字段：说明选择理由
- `answer` 字段：只包含选项字母（A/B/C/D）
- 必须是有效的 `JSON` 格式（可被 `json.loads()` 解析）
- **不要用 markdown 代码块包裹**（不要用 ````json` 或 `````）
- 直接输出纯 `JSON` 字符串，不要添加任何说明文字

个人实现

`{retrieved_knowledge}`

请基于以上检索到的硬件知识，回答以下选择题：

题目：{`problem`}

请严格遵循以下要求：

1. 仔细阅读题目和检索到的知识
2. 基于检索到的知识进行推理
3. 输出格式必须是标准的 `JSON`，包含 `reasoning` 和 `answer` 两个字段
4. `answer` 字段只能包含选项字母（A/B/C/D），不能有其他字符

你是一个严格的计算硬件知识问答助手。你的任务是根据检索到的相似硬件知识条目，解答计算硬件相关的单项选择题，并输出标准 `JSON`。

【任务要求】

1. 参考提供的相似硬件知识条目解释，分析并回答当前硬件选择题。
2. `reasoning` 字段：说明选择理由和判断依据。
3. `answer` 字段：仅包含正确的选项字母（A/B/C/D），不要包含任何其他字符。

【输出格式】

严格输出纯 `JSON` 字符串，必须包含 `reasoning` 和 `answer` 两个字段。
绝对不要包含 ````json` 或 ````` 等 `Markdown` 代码块，不要输出任何解释或多余字符！

【标准 `JSON` 示例】

```
{
  "reasoning": "GPU 擅长进行大规模并行计算，常用于图形渲染和并行任务，因此选择 A。",
  "answer": "A"
}
```

本地测试分数：50/50

例题：B1-三国知识整理助手（得满分较难）

原题题面

构建 System Prompt，引导模型从各种格式的三国历史文本中提取关键信息

要求：

1. 引导模型理解整理任务（输入：各种格式的历史文本 → 输出：标准三行格式）
2. 说明需要提取的三要素：时间、地点、人物
3. 要求严格按照标准三行格式输出
4. 强调要提取关键信息，忽略无关的修饰内容
5. 提供示例帮助模型理解任务

输出格式要求：

时间：【时间信息】

地点：【地点信息】

人物：【人物列表，用顿号”、”分隔】

注意：

- 必须恰好三行，不多不少
- 第一行以“时间：”开头
- 第二行以“地点：”开头
- 第三行以“人物：”开头
- 多个人物用中文顿号”、”分隔
- 不要添加 markdown 格式或代码块

个人实现

例题：B1-三国知识整理助手（得满分较难）

原题题面

构建 System Prompt，引导模型从各种格式的三国历史文本中提取关键信息。

要求：

1. 引导模型理解整理任务（输入：各种格式的历史文本 → 输出：标准三行格式）
2. 说明需要提取的三要素：时间、地点、人物
3. 要求严格按照标准三行格式输出
4. 强调要提取关键信息，忽略无关的修饰内容
5. 提供示例帮助模型理解任务

输出格式要求：

时间：【时间信息】
地点：【地点信息】
人物：【人物列表，用顿号”、”分隔】

注意：

- 必须恰好三行，不多不少
- 第一行以“时间：”开头
- 第二行以“地点：”开头
- 第三行以“人物：”开头
- 多个人物用中文顿号”、”分隔
- 不要添加 markdown 格式或代码块

个人实现

你是一个历史知识整理助手，专门从各种格式的三国历史文本中提取关键信息。

【任务要求】

1. 从输入的历史文本中提取三个核心要素：时间、地点、人物。
2. 时间规则：可以是具体年份（如“公元208年”）或时期（如“建安五年”、“东汉末年”）。
3. 地点规则：提取主要地名，去掉“在”、“于”等介词。
4. 人物规则：列出所有人名（不论故事中主要或非主要，败还是胜。虚构人物如貂蝉也要列出），不能漏掉任何一个！多个人物之间必须用中文顿号“、”分隔。

【输出格式】

严格输出恰好三行纯文本，不多不少，每行必须有内容：

时间：[时间信息]
地点：[地点信息]
人物：[人物列表，用顿号”、”分隔]

本地测试分数：30/30

绝对不要使用 Markdown 代码块（不要用“`”），绝对不要添加任何额外的说明文字！

【标准示例】

输入：建安十三年冬，孙刘联军在赤壁击败曹操大军，参加的将领有周瑜、程普、刘备等。

输出：

时间：建安十三年冬 这种格式表示，加上的话会对回答正确率有明显优化的 Prompt。
地点：赤壁
人物：曹操、周瑜、程普、刘备

例题：B2-带工具调用的数学计算助手（得满分较难）

原题题面

****支持的运算**:**

- 基本运算: `+` (加)、`-` (减)、`*` (乘)、`/` (除)
- 幂运算: `***` (Python 风格) 和 `^` (数学风格, 需要转换为`***`)
- 函数:
 - `sqrt(x)`: 平方根
 - `log(x)`: 以 10 为底的对数 (常用对数)
 - `ln(x)`: 以 e 为底的对数 (自然对数)
 - `sin(x)`, `cos(x)`, `tan(x)`: 三角函数 (x 为弧度)
 - `abs(x)`: 绝对值
- 常数:
 - `pi`: 圆周率 π
 - `e`: 自然常数 e

****评测流程** (自动进行, 考生不需要实现):**

1. ****第 1 轮****: 用户提问 $\rightarrow$ 模型生成工具调用 JSON
2. ****系统执行****: 提取表达式 $\rightarrow$ 调用 calculator $\rightarrow$ 得到结果
3. ****第 2 轮****: 系统返回结果 $\rightarrow$ 模型生成最终回复

****System Prompt 要求**:**

1. 引导模型理解用户的计算需求
2. 第 1 轮要求模型输出 JSON 格式, 包含 `tool_name` 和 `tool_expr` 两个字段
3. `tool_name` 固定为 `"calculator"`
4. `tool_expr` 为要计算的表达式 (字符串)
5. 说明表达式中可以使用的运算符、函数和常数
6. ****重要****: 明确说明第 2 轮拿到结果后要用 ****自然语言**** 回复, ****不要再输出 JSON****

****输出格式** (第 1 轮):**

```
'''json
{
  "tool_name": "calculator",
  "tool_expr": "3 * 4"
}
'''
```

【第 1 轮示例】

```
{
  "tool_name": "calculator",
  "tool_expr": "3 ** 4"
}
```

****System Prompt 设计要点**:**

1. ****明确角色****: 说明自己是计算助手, 需要使用工具
2. ****强调格式****: 第 1 轮必须输出 JSON, 包含 `tool_name` 和 `tool_expr` 两个字段
3. ****说明限制****: 不要直接计算, 必须调用工具
4. ****提供示例****: 给出具体的输出格式示例 (第 1 轮和第 2 轮都要有)
5. ****列出可用功能****: 说明支持的运算符、函数和常数
6. ****区分两轮格式****:
 - 第 1 轮: 输出 JSON 格式
 - 第 2 轮: 拿到结果后用 ****自然语言**** 回复, ****不要再输出 JSON****

个人实现

例题：B2-带工具调用的数学计算助手（得满分较难）

原题题面

****支持的运算**:**

- 基本运算: `+` (加)、`-` (减)、`*` (乘)、`/` (除)
- 幂运算: `***` (Python 风格) 和 `^` (数学风格, 需要转换为 `***`)
- 函数:
 - `sqrt(x)`: 平方根
 - `log(x)`: 以 10 为底的对数 (常用对数)
 - `ln(x)`: 以 e 为底的对数 (自然对数)
 - `sin(x)`, `cos(x)`, `tan(x)`: 三角函数 (x 为弧度)
 - `abs(x)`: 绝对值
- 常数:
 - `pi`: 圆周率 π
 - `e`: 自然常数 e

****评测流程** (自动进行, 考生不需要实现):**

- **第 1 轮**:** 用户提问 $\rightarrow$ 模型生成工具调用 JSON
- **系统执行**:** 提取表达式 $\rightarrow$ 调用 calculator $\rightarrow$ 得到结果
- **第 2 轮**:** 系统返回结果 $\rightarrow$ 模型生成最终回复

****System Prompt 要求**:**

- 引导模型理解用户的计算需求
- 第 1 轮要求模型输出 JSON 格式, 包含 `tool_name` 和 `tool_expr` 两个字段
- `tool_name` 固定为 `"calculator"`
- `tool_expr` 为要计算的表达式 (字符串)
- 说明表达式中可以使用的运算符、函数和常数
- **重要**:** 明确说明第 2 轮拿到结果后用 ****自然语言**** 回复, ****不要再输出 JSON****

****输出格式** (第 1 轮):**

```
'''json
{
  "tool_name": "calculator",
  "tool_expr": "3 * 4"
}
'''
```

【第 1 轮示例】

```
{
  "tool_name": "calculator",
  "tool_expr": "3 ** 4"
}
```

****System Prompt 设计要点**:**

- **明确角色**:** 说明自己是计算助手, 需要使用工具
- **强调格式**:** 第 1 轮必须输出 JSON, 包含 `tool_name` 和 `tool_expr` 两个字段
- **说明限制**:** 不要直接计算, 必须调用工具
- **提供示例**:** 给出具体的输出格式示例 (第 1 轮和第 2 轮都要有)
- **列出可用功能**:** 说明支持的运算符、函数和常数
- **区分两轮格式**:**
 - 第 1 轮: 输出 JSON 格式
 - 第 2 轮: 拿到结果后用 ****自然语言**** 回复, ****不要再输出 JSON****

个人实现

你是数学计算助手。你不能直接对数学计算进行口算, 必须通过调用工具来求解。

【任务要求】

- 遇到计算需求时, 必须在第 1 轮输出 **工具调用请求**, 严禁直接回答计算结果。
- 可用的计算表达式支持:
 - 运算符: `+` `-` `*` `/` `**` `^`
 - 函数: `sqrt`, `log`, `ln`, `sin`, `cos`, `tan`, `abs`
 - 常数: `pi`, `e`
 - **注意: 所有的角度必须用 pi 的有理数倍表示! 例如, 你必须将 60° 转换为 $\pi/3$! 必须将 45° 转换为 $\pi/4$! 必须将 30° 转换为 $\pi/6$!**
- 第 2 轮收到工具计算结果 x 后, 你必须直接输出 x, 用自然语言回答用户! 严禁再次输出 JSON!

【第 1 轮输出格式】

严格输出纯 JSON 字符串, 包含 `tool_name` (固定为 `"calculator"`) 和 `tool_expr` 两个字段。
绝对不要包含 `'''json` 等 Markdown 格式!

【第 1 轮示例】

```
{
  "tool_name": "calculator",
  "tool_expr": "3 ** 4"
}
```

【第 1 轮返回值示例】81

【第 2 轮示例】

计算结果为 81。

本地测试分数: 30/30

小结

总之，利用本节所讲的大致方法，我们就能以低门槛写出来一个较完美的 Prompt。在对细节进行微调后，我们就能在其他部分完善的情况下，获得几乎满分的好成绩。

借助此法，选手赛时仅需自主编写约 30% 的提示词内容，其余可直接参照题面（**我们只需创作图片中加粗的部分**），经少量练习即可熟练运用，兼顾了效率与完备性。

Outline

观前提示

何为选手视角

LMCC 第二轮常见知识点梳理

Prompt 编写方法

生成参数配置

Python 代码实现

尾声



生成参数配置

在 LMCC 中，常用的生成参数有三个：`max_new_tokens`、`enable_thinking` 和 `do_sample`。接下来，我们将从选手视角介绍它们的含义及赛时使用方法。

(这里我们的常用策略是设置 `do_sample=False`，因此就不展开 `temperature`、`top_k` 等参数了，建议参考其他老师的讲解。)

do_sample: 采样开关

do_sample: 若设为 `True`, 模型在每一步会从概率分布中随机抽取下一个 token, 所以每次运行输出可能不同。**此时模型更具随机性和创造力, 适合写诗、写文章等任务。**若设为 `False`, 模型会直接选择概率最高的那个 token 输出 (即贪心解码)。对于相同输入, 模型输出一定不变。**此时模型更具逻辑性, 适合 LMCC 第二轮中绝大多数题目。**

总之, 我们**对于大部分题目 (例如这里的六道题)**选择 `do_sample=False`, 不让模型过于“放飞自我”。但若遇到特殊题目 (如 `evaluate.py` 会对模型写的诗打分且要求分数尽量高), 则可酌情更改。

enable_thinking: 思考模式开关

enable_thinking: 若设为 `True`, 模型会先输出一段由 `<think></think>` 包裹的推理过程, 再输出最终答案。总体而言, 这能让模型变得更聪明, 也让基础模型有更多“发挥”空间, 有利有弊。**此时模型输出较慢, 一般适用于逻辑/流程复杂的任务。**若设为 `False`, 模型会直接输出最终答案, 不进行深度思考, 不让模型想太多。**此时模型输出较快, 适用于逻辑/流程简单的任务, 以保持输出的稳定性。**

在**调试**时, 可以用 `False` 让模型输出更快, 用 `True` 让模型更聪明; 在**正式使用**时, 优先用 `True` 开启思考。但如果问题简单, 开启思考后屡次答错 (我们的 AI 比较基础, 所以**有时开思考反而容易胡思乱想**), 那么**切换为 `False` 也是常见选择**。总之, **可将开 T/F 视为同一模型的两面 (参考 B2)**, 只需让其中一面完成题目所需的任务即可。

最后, 目前 LMCC 第二轮的大部分题目在评测时都会自动过滤 `<think></think>` 之间的文本。若赛时实在不放心, 可在 `evaluate.py` 源码中 `Ctrl+F` 搜索 `think`, 观察代码注释来验证。故一般情况下开启思考是合法的, 赛时可自行尝试。

小互动

此前我们已经接触过 A1、A2、B1、B2。重新阅读这些助手的期望输出与目标，大家猜猜**这些助手的 `enable_thinking` 在我们的方法下最好设为什么？**（T；F；T/F）
（PS：不必全部判断正确，有大体感觉即可；以下结果均来自实际测试。）

小互动

A1-数学作业批改助手

(任务) 读取学生作业内容, 逐题判断学生答案与标准答案是否一致, 并输出正确/错误列表。

(输出格式) JSON: {"student_id": "S001",
"judgements": [true, false, true]}

A2-硬件知识问答助手

(任务) 基于检索到的若干道相似历史题目, 分析解答当前硬件知识 A/B/C/D 选择题。

(输出格式) JSON: {"reasoning": "推理过程",
"answer": "C"}

B1-三国知识整理助手

(任务) 从各种格式的三国历史文本中, 分析并找出“时间、地点、人物”三要素, 提取整理为标准三行格式

(输出格式) 时间: 【时间信息】

地点: 【地点信息】

人物: 【人名1、人名2、人名3】

B2-带工具调用的数学计算助手 (任务) 第一轮输出计算题对应的JSON来调用计算器工具, 后端计算返回结果后, 第二轮用自然语言直接回复计算结果。

(输出格式) 第一轮 JSON: {"tool_name": "calculator",
"tool_expr": "2026+812"}

第二轮 自然语言: 计算结果为 2838

小互动

A1-数学作业批改助手

(任务) 读取学生作业内容, 逐题判断学生答案与标准答案是否一致, 并输出正确/错误列表。

(输出格式) JSON: {"student_id": "S001",
"judgements": [true, false, true]}
`enable_thinking = True / False?`

B1-三国知识整理助手

(任务) 从各种格式的三国历史文本中, 分析并找出“时间、地点、人物”三要素, 提取整理为标准三行格式

`enable_thinking = True / False?`

(输出格式) 时间: 【时间信息】

地点: 【地点信息】

人物: 【人名1、人名2、人名3】

A2-硬件知识问答助手

(任务) 基于检索到的若干道相似历史题目, 分析解答当前硬件知识 A/B/C/D 选择题。

(输出格式) JSON: {"reasoning": "推理过程",
"answer": "C"} `enable_thinking = True / False?`

B2-带工具调用的数学计算助手 (任务) 第一轮输出计算题对应的JSON来调用计算器工具, 后端计算返回结果后, 第二轮用自然语言直接回复计算结果。

(输出格式) 第一轮 JSON: {"tool_name": "calculator",
"tool_expr": "2026+812"}

第二轮 自然语言: 计算结果为 2838

`enable_thinking = True / False?`

小互动

A1-数学作业批改助手

(比较正常)

(任务) 读取学生作业内容, 逐题判断学生答案与标准答案是否一致, 并输出正确/错误列表。

(输出格式) JSON: {"student_id": "S001",

"judgements": [true, false, true]}

`enable_thinking = True / False?`

需要进行数学计算

并进行结果的比较。对基础的 AI, 这是逻辑复杂的

B1-三国知识整理助手

(比较正常)

(任务) 从各种格式的三国历史文本中, 分析并找出“时间、地点、人物”三要素, 提取整理为标准三行格式

`enable_thinking = True / False?`

(输出格式) 时间: 【时间信息】

需要从复杂的文本

地点: 【地点信息】

中寻找信息, 不开

人物: 【人名1、人名2、人名3】

思考的话, 容易漏掉关键内容

A2-硬件知识问答助手

(比较正常)

(任务) 基于检索到的若干道相似历史题目, 解答当前硬件知识 A/B/C/D 选择题。

(输出格式) JSON: {"reasoning": "推理过程",

"answer": "C"} `enable_thinking = True / False?`

文本信息比对, 不需要复杂逻辑, 查找即可。

所以由于任务逻辑简单, 可开可不开

B2-带工具调用的数学计算助手 (任务) 第一轮输出计算题对应的JSON来调用计算器工具, 后端计算返回结果

后, 第二轮用自然语言直接回复计算结果。 (例外情况)

(输出格式) 第一轮 JSON: {"tool_name": "calculator",

"tool_expr": "2026+812"}

第二轮 自然语言: 计算结果为 2838

`enable_thinking = True / False?` 这是一个例外。

第一轮和第二轮的任务都只需要拼凑信息, 实测第一轮没难度第二轮较难。开思考的话由于神秘原因会转写错

max_new_tokens: 最大生成长度

max_new_tokens: 表示模型在本次回复中最多生成多少个 **new (新) token**。模型输出的内容一旦超过这个数字，就会被直接截断，后面的全部丢弃。

下面是“新 token”的含义：

- **新**: 不包含输入 prompt 中的 token，只统计模型新生成的部分。
- **token**: 不是字符数，也不是字数。在中文里，一个 token 大约对应 1.5~2 个汉字。

一般来说，**只要 prompt 设定得足够好，就不需要刻意把 max_new_tokens 调小，除非赶输出时间**。因为在 `do_sample=False` 的情况下，`max_new_tokens` 较小的模型输出，本质上就是较大模型输出的截断版。

所以，可以先“无脑”**用常用值（如 256、512）去套**，再根据模型的实际输出**做调整**（例如**没识别到答案，可能是 max_new_tokens 设小了**）。另外，**遇到复杂任务时，可以酌情调高 max_new_tokens**。这逻辑简单，稍加尝试就能调对，我就不举例了。

小结

如果你看不太懂本节前面的内容，那就直接来看这里！我们用简单有趣的语言串一下：

- **do_sample**：除了写诗之类的极具创造性的题目，绝大多数题目**设为 False**。
新手开 **True** 要谨慎。
- **enable_thinking**：开 T 想得多、输出慢，适合复杂任务；开 F 想得少、输出快，适合简单任务，做得稳。**时间紧用 F，大部分题用 T**。不过**这里的千问**只有 0.6B，远没有很多人平时用的豆包、DeepSeek、ChatGPT 那些大模型聪明。比如 DeepSeek-V4-Pro 大概有 1000B-2000B。所以**有时开 T 在简单任务上反而没有 F 聪明，别掉坑里**。
- **max_new_tokens**：控制模型输出长度。先设 256、512 这类值，再根据评分程序有没有读到答案来调整（尽量往大调）。

Outline

观前提示

何为选手视角

LMCC 第二轮常见知识点梳理

Prompt 编写方法

生成参数配置

Python 代码实现

尾声



Python 代码实现

受篇幅所限，本节将**着重介绍 A2 中所涉及的 PyTorch 基本语法**，而 Python 的基础知识（如变量、条件判断等）**则由其他讲者另行介绍**。

对于初学者，若希望进一步学习或练习 Python，以下途径供参考（笔者个人建议）：

- 查阅 **官方文档（中文版）**。
- 利用“**类搜索引擎**”（如 **bing zhihu**）检索相关教程与资料。
- 在在线评测平台（OJ）上通过**刷题**来巩固编程能力。
- 针对语法细节或代码中的具体问题，可向较智能的 AI（如 **DeepSeek、Claude**）求助。但需注意模型可能产生“幻觉”，且不应仅仅机械复述其回答，而应在理解的基础上加以内化与记忆。
- **关注 CCF 主办的在线讲座**，如 LMCC 线上培训讲座。
- **参与 CCF 组织的 Python 等级考试（如 GESP）等相关活动**。
-

PyTorch 快速上手小课件

1. 何为 张量 (Tensor)

张量 (Tensor) 是 PyTorch 中的基本数据结构, 你可以把它看作是多维数组。几乎所有的数据、模型参数在 PyTorch 中都是以 Tensor 的形式存在的。

```
import torch
x = torch.tensor([1.0, 2.0, 3.0])
zeros = torch.zeros(2, 3)
ones = torch.ones(2, 3)
rand = torch.rand(2, 3)
print(x.shape)
```

创建一个普通的 Tensor
创建全 0、全 1 或随机矩阵
2行3列的全0矩阵
2行3列的全1矩阵
2行3列的随机数矩阵 (0~1之间)
查看每一维大小拼起来的 torch
这会输出 torch.Size([3])

PyTorch 快速上手小课件

2. Tensor 的运算

```
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
A = torch.norm(torch.tensor([3, 4])) # norm 即范数。这里即向量 (3,4) 的长度，勾三股四弦五，所以长度为 A=tensor(5.)。
c = a + b # 加法
d = a * b # 逐元素相乘（对应元素相乘）
e = torch.dot(a, b) # 点积 (1*4 + 2*5 + 3*6 = 32)
mat1 = torch.ones(2, 3)
mat2 = torch.ones(3, 2)
result = torch.matmul(mat1, mat2) # 矩阵乘法，结果为 2x2 矩阵
```

可以发现，norm dot ones matmul(matrix+mul) 这些名字基本都是英文的直接转写。

对于有基础的同学，这就很好记了。若无基础，熟能生巧，这也不难记忆。

PyTorch 快速上手小课件

3. 自动求导 (Autograd)

这是 PyTorch 成为深度学习框架的关键所在。在训练神经网络时，我们需要计算梯度来更新参数。PyTorch 可以自动帮我们完成这些微积分计算。

```
# requires_grad=True 告诉 PyTorch 追踪这个张量的所有操作，以便后续计算梯度
x = torch.tensor(2.0, requires_grad=True)

y = x ** 2 + 3 * x + 1 # 相当于 二次函数  $y = x^2 + 3x + 1$ 
y.backward()           # 自动计算梯度 ( $dy/dx$ )
# y 关于 x 的导数是  $2x + 3$ 。当  $x = 2$  时，梯度应该是  $2*2+3=7$ 。
print(x.grad)          # 输出: tensor(7.) (这里的 7. 是浮点数 7.0 的意思)
```

例题 (A2)

`def compute_similarity(query_embedding: torch.Tensor, doc_embedding: torch.Tensor) → float:` # 基于上述内容, 大家应已具备“拼”出完整代码的能力。不妨尝试回忆这涉及到的关键函数名

"""

考生实现: 计算两个向量的余弦相似度

参数:

query_embedding: 查询的 embedding 向量, shape: (embedding_dim,)

doc_embedding: 文档的 embedding 向量, shape: (embedding_dim,)

返回:

float: 余弦相似度, 范围 [-1, 1], 值越大表示越相似

"""

===== 考生实现区域 (可修改) =====

余弦相似度: $(A \cdot B) / (\|A\| * \|B\|)$

TODO: 实现余弦相似度计算

`return 0`

===== 考生实现区域 (可修改) =====

例题解答 (A2)

```
def compute_similarity(query_embedding: torch.Tensor, doc_embedding:
torch.Tensor) → float: # 注意这里的返回值是 float 而不是 tensor
    # 余弦相似度:  $(A \cdot B) / (||A|| * ||B||)$ 
    # 本质上要算的就是 query_embedding, doc_embedding 的点积, 然后除以两个范数, 也就是 norm
    #  $A / ||A||$  是归一化后的单位向量, 所以本质上这是在将向量归一化之后求点积, 这样理解就会自然一些
    dot_product = torch.dot(query_embedding, doc_embedding) # 求  $A \cdot B$ 
    norm_query = torch.norm(query_embedding) # 求  $||A||$ 
    norm_doc = torch.norm(doc_embedding) # 求  $||B||$ 
    if norm_query==0 or norm_doc==0: return 0.0 # 不能除以 0
    similarity = dot_product / (norm_query * norm_doc) # 求余弦相似度
    return similarity.item() # 注意要返回 tensor 里面的东西而不是 tensor。返回“盒子”里的数字而不是整个“盒子”
```


Outline

观前提示

何为选手视角

LMCC 第二轮常见知识点梳理

Prompt 编写方法

生成参数配置

Python 代码实现

尾声



尾声

今天，我们将 LMCC 第二轮的常用知识点分为了三大块：

Prompt 编写方法（重点）、生成参数配置、代码实现

并对每一部分进行了选手视角的浅谈。

希望“**选手视角**”带给大家的，不仅是实打实的实战应试经验，更是一种**比单纯接收理论更轻松、更具象的知识学习方式**。

尾声

今天，我们将 LMCC 第二轮的常用知识点分为了三大块：

Prompt 编写方法（重点）、生成参数配置、代码实现

并对每一部分进行了选手视角的浅谈。

希望“**选手视角**”带给大家的，不仅是实打实的实战应试经验，更是一种**比单纯接收理论更轻松、更具象的知识学习方式**。

笔者平时也比较喜欢写文章。特别地，公众号 **培淇的秘密基地** 中就放了我写过的大部分数学/OI 文章。**有答疑需求或建议的话可以私信**。这些文章中，个人觉得最有趣的一篇是关于**证明几乎所有偶数都满足哥德巴赫猜想**的。感兴趣的话，也可以去看看（）

尾声

今天，我们将 LMCC 第二轮的常用知识点分为了三大块：

Prompt 编写方法（重点）、生成参数配置、代码实现

并对每一部分进行了选手视角的浅谈。

希望“**选手视角**”带给大家的，不仅是实打实的实战应试经验，更是一种**比单纯接收理论更轻松、更具象的知识学习方式**。

笔者平时也比较喜欢写文章。特别地，公众号 **培淇的秘密基地** 中就放了我写过的大部分数学/OI 文章。**有答疑需求或建议的话可以私信**。这些文章中，个人觉得最有趣的一篇是关于**证明几乎所有偶数都满足哥德巴赫猜想**的。感兴趣的话，也可以去看看（）

最后，**Thank you for listening!**